

Porter Logistics Case Study

About Porter

Porter is India's Largest Marketplace for Intra-City Logistics. Leader in the country's \$40 billion intra-city logistics market, Porter strives to improve the lives of 1,50,000+ driver-partners by providing them with consistent earning & independence. Currently, the company has serviced 5+ million customers

Business Problem

Porter works with a wide range of restaurants for delivering their items directly to the people. Porter has a number of delivery partners available for delivering the food, from various restaurants and wants to get an estimated delivery time that it can provide the customers on the basis of what they are ordering, from where and also the delivery partners. Porter wants to estimate the delivery time for a given order and improve the customer experience by providing accurate delivery time estimates. The company has provided a dataset containing information about past orders, including the time taken for each delivery. The goal is to build a regression model that can predict the delivery time based on various features such as order details, restaurant information, and delivery partner availability. This dataset has the required data to train a regression model that will do the delivery time estimation, based on all those features

Metrics

We will use Mean Squared Error (MSE) as the evaluation metric for our regression model. MSE is a common metric used to measure the average squared difference between the predicted and actual values. A lower MSE indicates better performance of the model.

We will also use MAE (Mean Absolute Error) as an additional metric to evaluate the model's performance. MAE measures the average absolute difference between the predicted and actual values, providing a more interpretable measure of prediction accuracy.

Data Dictionary

Each row in this file corresponds to one unique delivery. Each column corresponds to a feature as explained below.

1. market_id : integer id for the market where the restaurant lies
2. created_at : the timestamp at which the order was placed
3. actual_delivery_time : the timestamp when the order was delivered
4. store_primary_category : category for the restaurant
5. order_protocol : integer code value for order protocol(how the order was placed ie: through porter, call to restaurant, pre booked, third part etc)
6. total_items subtotal : final price of the order
7. num_distinct_items : the number of distinct items in the order
8. min_item_price : price of the cheapest item in the order
9. max_item_price : price of the costliest item in order
10. total_onshift_partners : number of delivery partners on duty at the time order was placed
11. total_busy_partners : number of delivery partners attending to other tasks
12. total_outstanding_orders : total number of orders to be fulfilled at the moment

```
In [1]: import polars as pl
import duckdb as db
import numpy as np
from sklearn.impute import KNNImputer
from sklearn.preprocessing import StandardScaler

In [2]: df = pl.read_csv("../data/raw/dataset.csv")
```

Data Exploration

```
In [3]: df.head()
```

Out[3]: shape: (5, 14)

market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshif
f64	str	str	str	str	f64	i64	i64	i64	i64	i64	
1.0	"2015-02-06 22:24:17"	"2015-02-06 23:27:16"	"df263d996281d984952c07998dc543...	"american"	1.0	4	3441	4	557	1239	
2.0	"2015-02-10 21:49:25"	"2015-02-10 22:56:29"	"f0ade77b43923b38237db569b016ba...	"mexican"	2.0	1	1900	1	1400	1400	
3.0	"2015-01-22 20:39:28"	"2015-01-22 21:09:09"	"f0ade77b43923b38237db569b016ba...	null	1.0	1	1900	1	1900	1900	
3.0	"2015-02-03 21:21:45"	"2015-02-03 22:13:00"	"f0ade77b43923b38237db569b016ba...	null	1.0	6	6900	5	600	1800	
3.0	"2015-02-15 02:40:36"	"2015-02-15 03:20:26"	"f0ade77b43923b38237db569b016ba...	null	1.0	3	3900	3	1100	1600	

In [4]: df.shape

Out[4]: (197428, 14)

In [5]: df.describe()

Out[5]: shape: (9, 15)

statistic	market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_pri
str	f64	str	str	str	str	f64	f64	f64	f64	f64	f
"count"	196441.0	"197428"	"197421"	"197428"	"192668"	196433.0	197428.0	197428.0	197428.0	197428.0	197428.0
"null_count"	987.0	"0"	"7"	"0"	"4760"	995.0	0.0	0.0	0.0	0.0	(
"mean"	2.978706	null	null	null	null	2.882352	3.196391	2682.331402	2.670791	686.21847	1159.588
"std"	1.524867	null	null	null	null	1.503771	2.666546	1823.093688	1.630255	522.038648	558.4113
"min"	1.0	"2014-10-19 05:24:15"	"2015-01-21 15:58:11"	"0004d0b59e19461ff126e3a08a814c...	"afghan"	1.0	1.0	0.0	1.0	-86.0	(
"25%"	2.0	null	null	null	null	1.0	2.0	1400.0	1.0	299.0	800
"50%"	3.0	null	null	null	null	3.0	3.0	2200.0	2.0	595.0	1095
"75%"	4.0	null	null	null	null	4.0	4.0	3395.0	3.0	949.0	1395
"max"	6.0	"2015-02-18 06:00:44"	"2015-02-19 22:45:31"	"ffedf5be3a86e2ee281d54cdc97bc1...	"vietnamese"	7.0	411.0	27100.0	20.0	14700.0	14700

In [6]: dict(zip(df.columns, df.dtypes))

Out [6]: {'market_id': Float64, 'created_at': String, 'actual_delivery_time': String, 'store_id': String, 'store_primary_category': String, 'order_protocol': Float64, 'total_items': Int64, 'subtotal': Int64, 'num_distinct_items': Int64, 'min_item_price': Int64, 'max_item_price': Int64, 'total_onshift_partners': Float64, 'total_busy_partners': Float64, 'total_outstanding_orders': Float64}

In [7]: df.null_count()

Out [7]: shape: (1, 14)

market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshift_partners	total_busy_partners	total_outstanding_orders
u32	u32	u32	u32	u32	u32	u32	u32	u32	u32	u32	u32	u32	u32
987	0	7	0	4760	995	0	0	0	0	0	16262	162	162

In [8]: db.sql("""select * from df where total_onshift_partners IS NULL limit 20""")

Out [8]:

market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshift_partners	total_busy_partners	total_outstanding_orders
double	varchar	varchar	varchar	varchar	double	int64	int64	int64	double	double	double	double	double
int64	int64	double	double	double	double	double	double	double	double	double	double	double	double
225	6.0	2015-02-06 01:11:56	2015-02-06 01:42:51	45d38ce7f5231602e24a2103a0300ae6	breakfast	2.0	2	575	2	1415	3	1	1
185	6.0	2015-02-14 02:07:47	2015-02-14 03:17:37	45d38ce7f5231602e24a2103a0300ae6	breakfast	2.0	5	1415	3	1415	3	1	1
650	6.0	2015-01-31 21:58:30	2015-01-31 22:55:32	45d38ce7f5231602e24a2103a0300ae6	breakfast	2.0	1	650	1	650	1	1	1
225	6.0	2015-02-08 03:28:59	2015-02-08 05:32:11	45d38ce7f5231602e24a2103a0300ae6	breakfast	2.0	5	1550	5	1550	5	5	5
185	6.0	2015-01-23 19:29:17	2015-01-23 20:25:25	45d38ce7f5231602e24a2103a0300ae6	breakfast	2.0	6	1110	5	1110	5	5	5
150	6.0	2015-02-03 22:27:24	2015-02-03 23:09:31	45d38ce7f5231602e24a2103a0300ae6	breakfast	2.0	8	2115	8	2115	8	8	8
889	6.0	2015-01-31 01:35:23	2015-01-31 02:16:43	45d38ce7f5231602e24a2103a0300ae6	breakfast	2.0	2	1741	2	1741	2	2	2
225	6.0	2015-01-24 23:17:11	2015-01-25 00:24:49	45d38ce7f5231602e24a2103a0300ae6	breakfast	2.0	7	2250	2	2250	2	2	2
3900	6.0	2015-02-03 21:15:23	2015-02-03 22:38:32	45d38ce7f5231602e24a2103a0300ae6	breakfast	2.0	1	3900	1	3900	1	1	1
400	6.0	2015-02-15 19:57:36	2015-02-15 20:49:04	4a06d868d044c50af0cf9bc82d2fc19f	sandwich	1.0	4	3575	4	3575	4	4	4
979	6.0	2015-01-25 19:15:56	2015-01-25 19:40:38	4a06d868d044c50af0cf9bc82d2fc19f	sandwich	1.0	2	2374	2	2374	2	2	2
0	6.0	2015-01-28 19:50:12	2015-01-28 20:41:22	e143c01e314f7b950daca31188cb5d0f	sandwich	1.0	7	2503	4	2503	4	4	4
179	6.0	2015-01-28 21:34:15	2015-01-28 22:01:36	e143c01e314f7b950daca31188cb5d0f	sandwich	1.0	5	1997	5	1997	5	5	5
0	6.0	2015-02-17 19:38:04	2015-02-17 20:02:27	e143c01e314f7b950daca31188cb5d0f	sandwich	1.0	3	1274	3	1274	3	3	3
899	6.0	2015-01-29 21:18:12	2015-01-29 21:43:05	e143c01e314f7b950daca31188cb5d0f	sandwich	1.0	2	1798	1	1798	1	1	1
999	6.0	2015-02-14 21:20:04	2015-02-14 21:54:10	e143c01e314f7b950daca31188cb5d0f	sandwich	1.0	2	2497	2	2497	2	2	2
299	6.0	2015-02-03 23:33:16	2015-02-04 00:07:15	e143c01e314f7b950daca31188cb5d0f	sandwich	4.0	2	1597	2	1597	2	2	2
699	6.0	2015-02-07 03:40:27	2015-02-07 04:06:48	e143c01e314f7b950daca31188cb5d0f	sandwich	1.0	1	1048	1	1048	1	1	1
300	6.0	2015-02-07 21:28:56	2015-02-07 22:00:43	e143c01e314f7b950daca31188cb5d0f	sandwich	1.0	2	1199	2	1199	2	2	2
1095	6.0	2015-02-09 05:22:41	2015-02-09 05:51:35	1f0643149dd5c86751215fd0040675aa	korean	1.0	2	2390	2	2390	2	2	2
20 rows	14 columns												

In [9]: db.sql("""select * from df where store_primary_category IS NULL limit 20""")

market_id		created_at		actual_delivery_time		store_id		store_primary_category		order_protocol	total_items	subtotal	num_distinct_items	min_
item_price	max_item_price	total_onshift_partners	total_busy_partners	total_outstanding_orders										
double	varchar	varchar	varchar	varchar						double	int64	int64	int64	
int64	int64	double	double	double										
1900	3.0	2015-01-22 20:39:28	2015-01-22 21:09:09	f0ade77b43923b38237db569b016ba25	NULL				1.0	1	1900		1	
600	3.0	2015-02-03 21:21:45	2015-02-03 22:13:00	f0ade77b43923b38237db569b016ba25	NULL				1.0	6	6900		5	
1100	3.0	2015-02-15 02:40:36	2015-02-15 03:20:26	f0ade77b43923b38237db569b016ba25	NULL				1.0	3	3900		3	
1500	3.0	2015-01-28 20:30:38	2015-01-28 21:08:58	f0ade77b43923b38237db569b016ba25	NULL				1.0	3	5000		3	
1200	3.0	2015-01-31 02:16:36	2015-01-31 02:43:00	f0ade77b43923b38237db569b016ba25	NULL				1.0	2	3900		2	
750	3.0	2015-02-12 03:03:35	2015-02-12 03:36:20	f0ade77b43923b38237db569b016ba25	NULL				1.0	4	4850		4	
700	3.0	2015-02-18 01:15:45	2015-02-18 02:08:57	f0ade77b43923b38237db569b016ba25	NULL				1.0	2	2100		2	
1200	3.0	2015-02-02 19:22:53	2015-02-02 20:09:19	f0ade77b43923b38237db569b016ba25	NULL				4.0	4	4300		4	
600	3.0	2015-02-16 04:19:33	2015-02-16 06:34:00	f0ade77b43923b38237db569b016ba25	NULL				1.0	2	2200		2	
1900	3.0	2015-02-07 01:34:31	2015-02-07 02:17:14	f0ade77b43923b38237db569b016ba25	NULL				1.0	1	1900		1	
699	3.0	2015-01-25 01:50:51	2015-01-25 02:28:53	f0ade77b43923b38237db569b016ba25	NULL				4.0	4	4986		4	
275	1.0	2015-01-28 20:33:04	2015-01-28 21:04:14	50905d7b2216bfeccb5b41016357176b	NULL				NULL	3	1765		3	
975	1.0	2015-02-12 20:27:42	2015-02-12 21:18:29	50905d7b2216bfeccb5b41016357176b	NULL				2.0	1	975		1	
175	1.0	2015-01-28 21:37:38	2015-01-28 22:11:53	50905d7b2216bfeccb5b41016357176b	NULL				2.0	3	1425		3	
825	1.0	2015-01-21 20:35:27	2015-01-21 21:01:26	50905d7b2216bfeccb5b41016357176b	NULL				2.0	2	1750		2	
825	1.0	2015-02-16 19:19:17	2015-02-16 20:16:43	50905d7b2216bfeccb5b41016357176b	NULL				2.0	8	7870		6	
825	1.0	2015-01-27 20:26:19	2015-01-27 21:03:53	50905d7b2216bfeccb5b41016357176b	NULL				2.0	2	1725		2	
395	1.0	2015-01-22 20:19:08	2015-01-22 21:07:43	50905d7b2216bfeccb5b41016357176b	NULL				2.0	3	2015		3	
825	1.0	2015-02-10 20:05:17	2015-02-10 20:34:43	50905d7b2216bfeccb5b41016357176b	NULL				2.0	1	1220		1	
194	3.0	2015-01-23 19:22:18	2015-01-23 19:48:50	51b7dae1031b20174cacc7e69d6e4bf0	NULL				4.0	3	582		1	

```
db.sql("""
select * from df where order_protocol IS NULL limit 20
""")
```

id	market_id		created_at		actual_delivery_time		store_id		store_primary_category		order_protocol	total_items	subtotal	num_distinct_items	min_item_price
	item_price	max_item_price	total_onshift_partners	total_busy_partners	total_outstanding_orders	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items					
	double	varchar	varchar	varchar	varchar	varchar	double	int64	int64	int64					
	int64	int64	double	double	double	double	double	int64	int64	int64					
275	1.0	2015-01-28 20:33:04	2015-01-28 21:04:14	50905d7b2216bfeccb5b41016357176b	NULL	NULL	NULL	3	1765	3					
325	4.0	2015-01-24 19:48:45	2015-01-24 20:31:14	a87ff679a2f3e71d9181a67b7542122c	mediterranean	NULL	NULL	2	2070	2					
379	2.0	2015-02-10 19:36:38	2015-02-10 20:24:14	26324d8e2cc1957b8e581568a089a51c	NULL	NULL	NULL	9	4781	8					
1000	NULL	2015-02-17 02:17:43	2015-02-17 03:15:14	fe8c15fed5f808006ce95eddb7366e35	NULL	NULL	NULL	3	3400	3					
1295	4.0	2015-02-09 02:10:29	2015-02-09 02:59:20	6ea9ab1baa0efb9e19094440c317e21b	NULL	NULL	NULL	2	2790	2					
1995	5.0	2015-01-30 20:27:37	2015-01-30 21:17:51	b597976c3ce6012f3a07e9f5c71a3c8c	NULL	NULL	NULL	1	1995	1					
400	3.0	2015-01-23 02:21:51	2015-01-23 03:29:52	ed71773d43d53fa70ecf593c6582d9cc	sushi	NULL	NULL	6	5275	4					
1399	NULL	2015-02-17 03:49:46	2015-02-17 04:21:27	86311dbe35f1b6c5166365165602f54d	pizza	NULL	NULL	1	1699	1					
595	NULL	2015-02-07 02:11:12	2015-02-07 03:10:11	29cb2d8a436d82bf5dbcca7ea09edc5c	NULL	NULL	NULL	5	5240	5					
920	NULL	2015-01-23 04:09:03	2015-01-23 05:14:00	9f8684e630c4c30cad7b1f0935cd62ab	NULL	NULL	NULL	2	2240	1					
775	4.0	2015-02-10 23:44:10	2015-02-11 00:34:39	cf2226ddd41b1a2d0ae51dab54d32c36	fast	NULL	NULL	2	1850	1					
189	4.0	2015-01-29 01:09:59	2015-01-29 02:01:41	cf2226ddd41b1a2d0ae51dab54d32c36	NULL	NULL	NULL	3	1967	3					
160	NULL	2015-01-29 02:46:37	2015-01-29 03:47:41	fd8c07a31f8a85910ad8476f5f7efb27	chinese	NULL	NULL	2	890	2					
2575	2.0	2015-02-16 02:05:20	2015-02-16 02:44:08	0bb0846327772451045bd30dd347821b	NULL	NULL	NULL	1	2575	1					
200	1.0	2015-02-07 03:42:08	2015-02-07 05:00:30	35b47299c1130953d286c976e9c608dc	british	NULL	NULL	2	1750	2					
500	NULL	2015-01-21 20:42:48	2015-01-21 21:20:59	58e4d44e550d0f7ee0a23d6b02d9b0db	american	NULL	NULL	3	2150	2					
175	5.0	2015-02-15 18:24:36	2015-02-15 18:50:49	bb838bff2c2120b9237cac812c1019f8	NULL	NULL	NULL	6	1670	4					
100	3.0	2015-02-07 23:05:03	2015-02-07 23:35:05	8de0c3085da54b8e957220b9c8de8aca	NULL	NULL	NULL	6	3538	3					
1099	2.0	2015-01-24 05:14:44	2015-01-24 05:55:35	34be90676412d14c80f85809d8e3b97b	NULL	NULL	NULL	4	4696	2					
695	1.0	2015-01-29 00:57:22	2015-01-29 01:40:19	f187a23c3ee681ef6913f31fd6d6446b	pizza	NULL	NULL	1	995	1					

```
| 20 rows
14 columns |
```

```
db.sql("""
select market_id, store_primary_category, count(*) from df group by market_id, store_primary_category order by count(*) desc limit 20
""")
```


Out[11]:

market_id double	store_primary_category varchar	count_star() int64
2.0	mexican	6647
2.0	american	5700
1.0	american	5228
2.0	pizza	4859
1.0	pizza	4081
4.0	pizza	3840
2.0	sandwich	3751
2.0	burger	3340
4.0	mexican	3250
2.0	dessert	3115
4.0	chinese	2847
3.0	american	2743
1.0	japanese	2628
4.0	burger	2603
4.0	dessert	2601
4.0	vietnamese	2541
3.0	mexican	2487
1.0	mexican	2439
4.0	japanese	2403
5.0	american	2386
20 rows		3 columns

In [12]:

df["store_primary_category"].value_counts().sort("count", descending=True)

Out[12]: shape: (75, 2)

store_primary_category	count
str	u32
"american"	19399
"pizza"	17321
"mexican"	17099
"burger"	10958
"sandwich"	10060
...	...
"lebanese"	9
"belgian"	2
"indonesian"	2
"alcohol-plus-food"	1
"chocolate"	1

In [13]:

df["order_protocol"].value_counts().sort("count", descending=True)

Out[13]: shape: (8, 2)

order_protocol	count
f64	u32
1.0	54725
3.0	53199
5.0	44290
2.0	24052
4.0	19354
null	995
6.0	794
7.0	19

In [14]:

df["market_id"].value_counts().sort("count", descending=True)

Out[14]: shape: (7, 2)

market_id	count
f64	u32
2.0	55058
4.0	47599
1.0	38037
3.0	23297
5.0	18000
6.0	14450
null	987

In [15]:

df=df.with_columns(
 pl.col("min_item_price").abs(),
 pl.col("max_item_price").abs(),
 pl.col("total_onshift_partners").abs(),
 pl.col("total_busy_partners").abs(),
 pl.col("total_outstanding_orders").abs(),
)

In [16]:

df.describe()

Out[16]: shape: (9, 15)

statistic	market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_pri
str	f64	str	str	str	str	f64	f64	f64	f64	f64	f
"count"	196441.0	"197428"	"197421"	"197428"	"192668"	196433.0	197428.0	197428.0	197428.0	197428.0	197428
"null_count"	987.0	"0"	"7"	"0"	"4760"	995.0	0.0	0.0	0.0	0.0	(
"mean"	2.978706	null	null	null	null	2.882352	3.196391	2682.331402	2.670791	686.222339	1159.588
"std"	1.524867	null	null	null	null	1.503771	2.666546	1823.093688	1.630255	522.033561	558.4113
"min"	1.0	"2014-10-19 05:24:15"	"2015-01-21 15:58:11"	"0004d0b59e19461ff126e3a08a814c...	"afghan"	1.0	1.0	0.0	1.0	0.0	(
"25%"	2.0	null	null	null	null	1.0	2.0	1400.0	1.0	299.0	800
"50%"	3.0	null	null	null	null	3.0	3.0	2200.0	2.0	595.0	1095
"75%"	4.0	null	null	null	null	4.0	4.0	3395.0	3.0	949.0	1395
"max"	6.0	"2015-02-18 06:00:44"	"2015-02-19 22:45:31"	"ffedf5be3a86e2ee281d54cdc97bc1...	"vietnamese"	7.0	411.0	27100.0	20.0	14700.0	14700

In [17]: df.null_count()

Out[17]: shape: (1, 14)

market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshift_partners	total_busy_partners
u32	u32	u32	u32	u32	u32	u32	u32	u32	u32	u32	u32	u32
987	0	7	0	4760	995	0	0	0	0	0	16262	162

Null Values Imputation

In [18]: df = df.drop_nulls(subset=["total_onshift_partners", "total_busy_partners", "total_outstanding_orders", "actual_delivery_time",\
"market_id", "store_primary_category", "order_protocol"\
])

In [19]: df.describe()

Out[19]: shape: (9, 15)

statistic	market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price
str	f64	str	str	str	str	f64	f64	f64	f64	f64	f64
"count"	180240.0	"181159"	"181159"	"181159"	"176944"	180242.0	181159.0	181159.0	181159.0	181159.0	181159.0
"null_count"	919.0	"0"	"0"	"0"	"4215"	917.0	0.0	0.0	0.0	0.0	0.0
"mean"	2.748702	null	null	null	null	2.895923	3.208469	2698.524296	2.677659	684.854095	1160.7526
"std"	1.331715	null	null	null	null	1.514983	2.673052	1829.304441	1.627291	521.286542	561.8386
"min"	1.0	"2015-01-21 15:22:03"	"2015-01-21 15:58:11"	"0004d0b59e19461ff126e3a08a814c...	"afghan"	1.0	1.0	0.0	1.0	0.0	0.0
"25%"	2.0	null	null	null	null	1.0	2.0	1415.0	2.0	299.0	79
"50%"	2.0	null	null	null	null	3.0	3.0	2225.0	2.0	595.0	109
"75%"	4.0	null	null	null	null	4.0	4.0	3411.0	3.0	942.0	139
"max"	6.0	"2015-02-18 06:00:44"	"2015-02-19 22:45:31"	"ffeabd223de0d4eacb9a3e6e53e544...	"vietnamese"	7.0	411.0	26800.0	20.0	14700.0	1470

In [20]: db.sql("""
select * from df where max_item_price = 0 and min_item_price = 0
""")

Out[20]:

market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price
double	varchar	varchar	varchar	varchar	double	int64	int64	int64	double
int64	int64	double	double	double	double	int64	int64	int64	double
1.0	2015-02-09 18:23:31	2015-02-09 19:31:54	3e91970f771a2c473ae36b60d1146068	mexican	1.0	1	575	1	1.0
1.0	2015-01-22 17:57:08	2015-01-22 18:54:37	3e91970f771a2c473ae36b60d1146068	mexican	1.0	1	575	1	1.0
1.0	2015-02-13 18:41:31	2015-02-13 19:30:39	3e91970f771a2c473ae36b60d1146068	mexican	1.0	1	575	1	1.0
1.0	2015-01-21 19:36:57	2015-01-21 20:23:45	3e91970f771a2c473ae36b60d1146068	mexican	1.0	1	560	1	1.0
1.0	2015-02-13 02:37:10	2015-02-13 03:13:44	3e91970f771a2c473ae36b60d1146068	dim-sum	5.0	1	595	1	1.0
5.0	2015-01-27 02:26:45	2015-01-27 02:56:56	648f4baa45889f9c5f4f7add35862841	pizza	3.0	1	2418	1	1.0
1.0	2015-01-21 19:50:42	2015-01-21 20:24:07	da5e8bfed9bdb84595be92afeb3fd378	sandwich	1.0	2	1090	2	1.0

In [21]: df=df.with_columns(
pl.when(pl.col("max_item_price") == 0).then(pl.col("max_item_price").mean()).otherwise(pl.col("max_item_price")).alias("max_item_price"),
pl.when(pl.col("min_item_price") == 0).then(pl.col("min_item_price").mean()).otherwise(pl.col("min_item_price")).alias("min_item_price")
)

In [22]: db.sql("""
select * from df where max_item_price < min_item_price
""")

Out [22]:

	market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min
	item_price	max_item_price	total_onshift_partners	total_busy_partners	total_outstanding_orders					
	double	varchar	varchar	varchar	varchar	double	int64	int64	int64	
	double	double	double	double						
	4.0	2015-01-24 03:03:18	2015-01-24 04:02:30	a87ff679a2f3e71d9181a67b7542122c	mexican	1.0	1	1403	1	
1314.0	1292.0	129.0	129.0	127.0	205.0					
	1.0	2015-02-03 03:36:16	2015-02-03 05:13:04	c56a4706337730e0e15da875405fa1c5	fast	4.0	7	1223	5	684.
8540950215005	649.0	22.0	22.0	22.0	48.0					
	1.0	2015-01-25 21:38:45	2015-01-25 22:14:22	c56a4706337730e0e15da875405fa1c5	fast	4.0	6	958	6	684.
8540950215005	599.0	17.0	9.0	9.0	9.0					
	5.0	2015-01-31 20:10:47	2015-01-31 20:49:29	b597976c3ce6012f3a07e9f5c71a3c8c	NULL	1.0	1	1289	1	
1437.0	1400.0	21.0	13.0	13.0	19.0					
	5.0	2015-01-30 02:20:58	2015-01-30 03:14:42	b597976c3ce6012f3a07e9f5c71a3c8c	NULL	1.0	1	1828	1	
2031.0	1935.0	24.0	21.0	21.0	23.0					
	2.0	2015-01-27 04:09:54	2015-01-27 04:56:33	f806c5d2707545d718717be03e69a8d4	fast	4.0	14	1305	3	684.
8540950215005	190.0	65.0	93.0	93.0	77.0					
	2.0	2015-02-03 03:22:21	2015-02-03 04:14:00	f806c5d2707545d718717be03e69a8d4	fast	4.0	13	985	4	684.
8540950215005	390.0	96.0	89.0	89.0	154.0					
	5.0	2015-02-08 03:42:21	2015-02-08 04:20:11	f806c5d2707545d718717be03e69a8d4	nepalese	1.0	31	2820	7	684.
8540950215005	355.0	116.0	120.0	120.0	184.0					
	2.0	2015-01-31 19:28:49	2015-01-31 20:17:29	f806c5d2707545d718717be03e69a8d4	fast	4.0	40	3000	2	684.
8540950215005	150.0	48.0	45.0	45.0	51.0					
	2.0	2015-02-07 05:47:42	2015-02-07 06:26:49	f806c5d2707545d718717be03e69a8d4	fast	4.0	26	2320	5	684.
8540950215005	250.0	39.0	43.0	43.0	67.0					
	.	.	.	.	.	.	.	.	.	.
	.	.	.	.	.	.	.	.	.	.
	.	.	.	.	.	.	.	.	.	.
	.	.	.	.	.	.	.	.	.	.
	.	.	.	.	.	.	.	.	.	.
	3.0	2015-02-06 20:06:59	2015-02-06 20:40:50	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	1	1299	1	
805.0	765.0	6.0	6.0	6.0	9.0					
	4.0	2015-02-03 20:36:37	2015-02-03 21:04:49	4cc39b42f22ee5987819b83fc055cb33	pizza	3.0	1	625	1	
553.0	540.0	31.0	35.0	35.0	31.0					
	4.0	2015-02-07 20:52:15	2015-02-07 21:28:58	4cc39b42f22ee5987819b83fc055cb33	pizza	3.0	1	3174	1	
1513.0	1403.0	38.0	31.0	31.0	32.0					
	4.0	2015-02-16 02:34:06	2015-02-16 03:20:42	17e62166fc8586dfa4d1bc0e1742c08b	vietnamese	5.0	2	2086	1	
904.0	878.0	85.0	77.0	77.0	152.0					
	4.0	2015-02-03 02:57:44	2015-02-03 03:41:32	17e62166fc8586dfa4d1bc0e1742c08b	vietnamese	5.0	2	2082	2	
1020.0	928.0	88.0	87.0	87.0	150.0					
	1.0	2015-02-15 22:15:36	2015-02-15 22:42:14	1763ea5a7e72dd7ee64073c2dda7a7a8	fast	4.0	9	946	5	684.
8540950215005	299.0	24.0	17.0	17.0	17.0					
	1.0	2015-02-09 05:16:26	2015-02-09 05:48:49	1763ea5a7e72dd7ee64073c2dda7a7a8	fast	4.0	6	348	3	684.
8540950215005	169.0	20.0	20.0	20.0	18.0					
	1.0	2015-01-25 03:06:25	2015-01-25 03:32:31	1763ea5a7e72dd7ee64073c2dda7a7a8	fast	4.0	11	1351	5	684.
8540950215005	169.0	75.0	66.0	66.0	83.0					
	1.0	2015-01-30 19:24:55	2015-01-30 20:03:21	a914ecef9c12ffdb9bede64bb703d877	fast	4.0	1	1142	1	
737.0	701.0	22.0	19.0	19.0	32.0					
	1.0	2015-02-09 20:32:27	2015-02-09 21:09:10	a914ecef9c12ffdb9bede64bb703d877	fast	4.0	1	437	1	
497.0	448.0	36.0	26.0	26.0	27.0					
1778 rows (20 shown)										
14 columns										

```
In [23]: df=df.with_columns(  
    max_price_temp=pl.col('max_item_price'),  
    min_price_temp=pl.col('min_item_price'),  
)  
  
In [24]: df=df.with_columns(  
    max_item_price=pl.when(pl.col('max_price_temp') < pl.col('min_price_temp'))  
        .then(pl.col('min_price_temp'))  
        .otherwise(pl.col('max_price_temp')),  
    min_item_price=pl.when(pl.col('min_price_temp') > pl.col('max_price_temp'))  
        .then(pl.col('max_price_temp'))  
        .otherwise(pl.col('min_price_temp'))  
).drop('max_price_temp', 'min_price_temp')  
  
In [25]: db.sql("""  
    select * from df where  total_onshift_partners = 0  
    """)
```

Out[25]:

market_id	item_price	created_at		actual_delivery_time		store_id		store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_
		max_item_price	total_onshift_partners	total_busy_partners	total_outstanding_orders								
		double	varchar	varchar	varchar								
		double	double	double	double								
1.0	2015-01-27 02:29:00	2015-01-27 03:30:45	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	1	1150	1					
1150.0	1.0	2015-02-08 00:56:26	2015-02-08 01:50:48	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	4	4805	2				
1095.0	1.0	2015-01-24 04:01:10	2015-01-24 04:28:57	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	7	5200	5				
225.0	1.0	2015-02-01 06:20:26	2015-02-01 06:55:26	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	3	2910	3				
595.0	1.0	2015-02-14 03:10:56	2015-02-14 04:12:41	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	2	2495	2				
1095.0	1.0	2015-02-04 19:28:18	2015-02-04 20:11:13	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	2	2345	2				
1150.0	1.0	2015-01-25 03:22:11	2015-01-25 04:14:22	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	1	1250	1				
1150.0	1.0	2015-02-08 23:06:13	2015-02-08 23:42:58	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	2	1190	2				
595.0	1.0	2015-01-23 02:45:55	2015-01-23 03:32:26	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	7	5035	5				
195.0	1.0	2015-01-23 05:03:26	2015-01-23 05:56:30	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	3	2065	3				
195.0	.	.	.	.	.	.	.	.	.				
.	.	.	.	.	.	.	.	.	.				
.	.	.	.	.	.	.	.	.	.				
.	.	.	.	.	.	.	.	.	.				
.	.	.	.	.	.	.	.	.	.				
3.0	2015-01-28 16:18:30	2015-01-28 17:33:30	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	2	1640	2					
615.0	3.0	2015-02-17 19:03:53	2015-02-17 19:51:14	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	3	2500	3				
500.0	3.0	2015-02-14 17:31:55	2015-02-14 18:24:49	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	3	2700	3				
650.0	3.0	2015-02-10 19:03:48	2015-02-10 20:26:52	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	3	1415	3				
245.0	3.0	2015-02-12 21:07:28	2015-02-12 21:55:41	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	3	1415	3				
245.0	3.0	2015-01-23 17:53:47	2015-01-23 18:42:14	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	1	975	1				
975.0	3.0	2015-02-07 15:31:33	2015-02-07 16:24:02	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	5	2430	4				
270.0	3.0	2015-02-08 16:11:00	2015-02-08 17:36:23	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	4	2960	4				
480.0	3.0	2015-01-21 17:13:16	2015-01-21 17:51:09	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	2	1175	2				
345.0	3.0	2015-01-24 05:56:06	2015-01-24 06:27:21	10e4d7889812f78893b86aeb04111871	mexican	3.0	1	2599	1				
2599.0													
3615 rows (20 shown)													
14 columns													

In [26]:

```
db.sql("""
    select * from df where total_busy_partners = 0 and total_onshift_partners = 0 and total_outstanding_orders = 0
    """)
```

Out[26]:

market_id	item_price	created_at		actual_delivery_time		store_id		store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_
		max_item_price	total_onshift_partners	total_busy_partners	total_outstanding_orders								
		double	varchar	varchar	varchar								
		double	double	double	double								
1150.0	1.0	2015-01-27 1150.0	02:29:00	2015-01-27 03:30:45	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	1	1150	1			
1095.0	1.0	2015-02-08 1195.0	00:56:26	2015-02-08 01:50:48	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	4	4805	2			
225.0	1.0	2015-01-24 1095.0	04:01:10	2015-01-24 04:28:57	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	7	5200	5			
595.0	1.0	2015-02-01 1095.0	06:20:26	2015-02-01 06:55:26	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	3	2910	3			
1095.0	1.0	2015-02-14 1150.0	03:10:56	2015-02-14 04:12:41	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	2	2495	2			
1150.0	1.0	2015-02-04 1195.0	19:28:18	2015-02-04 20:11:13	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	2	2345	2			
1150.0	1.0	2015-01-25 1150.0	03:22:11	2015-01-25 04:14:22	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	1	1250	1			
595.0	1.0	2015-02-08 595.0	23:06:13	2015-02-08 23:42:58	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	2	1190	2			
195.0	1.0	2015-01-23 1150.0	02:45:55	2015-01-23 03:32:26	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	7	5035	5			
195.0	1.0	2015-01-23 1150.0	05:03:26	2015-01-23 05:56:30	5a01f0597ac4bdf35c24846734ee9a76	japanese	1.0	3	2065	3			
.	.	.	.	.	.	.	.	.	.	.			
.	.	.	.	.	.	.	.	.	.	.			
.	.	.	.	.	.	.	.	.	.	.			
.	.	.	.	.	.	.	.	.	.	.			
480.0	3.0	2015-01-24 950.0	17:11:50	2015-01-24 18:29:10	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	3	2455	3			
615.0	3.0	2015-01-28 925.0	16:18:30	2015-01-28 17:33:30	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	2	1640	2			
500.0	3.0	2015-02-17 975.0	19:03:53	2015-02-17 19:51:14	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	3	2500	3			
650.0	3.0	2015-02-14 975.0	17:31:55	2015-02-14 18:24:49	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	3	2700	3			
245.0	3.0	2015-02-10 810.0	19:03:48	2015-02-10 20:26:52	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	3	1415	3			
245.0	3.0	2015-02-12 810.0	21:07:28	2015-02-12 21:55:41	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	3	1415	3			
975.0	3.0	2015-01-23 975.0	17:53:47	2015-01-23 18:42:14	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	1	975	1			
270.0	3.0	2015-02-07 790.0	15:31:33	2015-02-07 16:24:02	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	5	2430	4			
480.0	3.0	2015-02-08 925.0	16:11:00	2015-02-08 17:36:23	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	4	2960	4			
345.0	3.0	2015-01-21 790.0	17:13:16	2015-01-21 17:51:09	dfb61b74af460c2fd68bb8266f9f0814	breakfast	1.0	2	1175	2			
3415 rows (20 shown)													
14 columns													

In [27]:

```
df=df.filter(
    ~(pl.col("total_onshift_partners") == 0)
)
```

In [28]:

```
df.describe()
```


Out [28]: shape: (9, 15)

	statistic	market_id	created_at	actual_delivery_time		store_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_pri
	str	f64	str	str		str	str	f64	f64	f64	f64	f64	ff
	"count"	176642.0	"177544"	"177544"		"177544"	"173383"	176640.0	177544.0	177544.0	177544.0	177544.0	177544
	"null_count"	902.0	"0"	"0"		"0"	"4161"	904.0	0.0	0.0	0.0	0.0	C
	"mean"	2.767864	null	null		null	null	2.898687	3.205588	2701.713919	2.675618	692.093454	1164.9864
	"std"	1.329898	null	null		null	null	1.513229	2.67464	1829.211488	1.62441	516.496735	559.2193
	"min"	1.0	"2015-01-21 15:22:03"	"2015-01-21 15:58:11"	"0004d0b59e19461ff126e3a08a814c...		"afghan"	1.0	1.0	0.0	1.0	1.0	52
	"25%"	2.0	null	null		null	null	1.0	2.0	1420.0	2.0	300.0	799
	"50%"	2.0	null	null		null	null	3.0	3.0	2228.0	2.0	599.0	1095
	"75%"	4.0	null	null		null	null	4.0	4.0	3421.0	3.0	943.0	1397
	"max"	6.0	"2015-02-18 06:00:44"	"2015-02-19 22:45:31"	"ffeabd223de0d4eacb9a3e6e53e544...		"vietnamese"	7.0	411.0	26800.0	20.0	14700.0	14700

```
In [29]: category_per_store = df.groupby("store_id").agg(
        pl.col("store_primary_category").mode().first().alias("store_primary_category_name")
    )
category_per_store = category_per_store.fill_null("other")
```

```
In [30]: df = df.join(category_per_store, on="store_id", how="left")
```

```
In [31]: df = df.with_columns(
        pl.when(pl.col("store_primary_category").is_null()).then(pl.col("store_primary_category_name")).otherwise(pl.col("store_primary_category")).alias("store_primary_category")
    ).drop("store_primary_category_name")
```

```
In [32]: market_id_per_store = df.groupby("store_id").agg(
        pl.col("market_id").mode().first().alias("market_id_num")
    )
market_id_per_store = market_id_per_store.fill_null(pl.col("market_id_num").mode().first())
```

```
In [33]: df = df.join(market_id_per_store, on="store_id", how="left")
```

```
In [34]: df = df.with_columns(
        pl.when(pl.col("market_id").is_null()).then(pl.col("market_id_num")).otherwise(pl.col("market_id")).alias("market_id")
    ).drop("market_id_num")
```

```
In [35]: df.null_count()
```

Out [35]: shape: (1, 14)

market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshift_partners	total_busy_partners
u32	u32	u32	u32	u32	u32	u32	u32	u32	u32	u32	u32	u32
0	0	0	0	0	0	904	0	0	0	0	0	0

```
In [36]: df = df.with_columns(
        pl.col("created_at").str.to_datetime(),
        pl.col("actual_delivery_time").str.to_datetime()
    )
```

Time based feature generation

```
In [40]: df=df.with_columns(
        pl.col("created_at").dt.offset_by('-5h30m'),
        pl.col("actual_delivery_time").dt.offset_by('-5h30m')
    )
```

```
In [41]: df = df.with_columns(
        pl.col("created_at").dt.year().alias("created_at_year"),
        pl.col("created_at").dt.month().alias("created_at_month"),
        pl.col("created_at").dt.day().alias("created_at_day"),
        pl.col("created_at").dt.hour().alias("created_at_hour"),
        pl.col("created_at").dt.minute().alias("created_at_minute"),
        pl.col("created_at").dt.second().alias("created_at_second"),
        pl.col("actual_delivery_time").dt.year().alias("actual_delivery_time_year"),
        pl.col("actual_delivery_time").dt.month().alias("actual_delivery_time_month"),
        pl.col("actual_delivery_time").dt.day().alias("actual_delivery_time_day"),
        pl.col("actual_delivery_time").dt.hour().alias("actual_delivery_time_hour"),
        pl.col("actual_delivery_time").dt.minute().alias("actual_delivery_time_minute"),
        pl.col("actual_delivery_time").dt.second().alias("actual_delivery_time_second")
    )
```

```
In [42]: df
```

Out [42]: shape: (177,544, 26)

market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshi
f64	datetime[μs]	datetime[μs]	str	str	f64	i64	i64	i64	f64	f64	
1.0	2015-02-06 16:54:17	2015-02-06 17:57:16	"df263d996281d984952c07998dc543...	"american"	1.0	4	3441	4	557.0	1239.0	
2.0	2015-02-10 16:19:25	2015-02-10 17:26:29	"f0ade77b43923b38237db569b016ba...	"mexican"	2.0	1	1900	1	1400.0	1400.0	
3.0	2015-01-22 15:09:28	2015-01-22 15:39:09	"f0ade77b43923b38237db569b016ba...	"other"	1.0	1	1900	1	1900.0	1900.0	
3.0	2015-02-03 15:51:45	2015-02-03 16:43:00	"f0ade77b43923b38237db569b016ba...	"other"	1.0	6	6900	5	600.0	1800.0	
3.0	2015-02-14 21:10:36	2015-02-14 21:50:26	"f0ade77b43923b38237db569b016ba...	"other"	1.0	3	3900	3	1100.0	1600.0	
...	...	...	...	...	...	...	...	...	...	...	...
1.0	2015-02-16 18:49:41	2015-02-16 19:54:48	"a914ecef9c12ffdb9bede64bb703d8...	"fast"	4.0	3	1389	3	345.0	649.0	
1.0	2015-02-12 18:31:59	2015-02-12 19:28:22	"a914ecef9c12ffdb9bede64bb703d8...	"fast"	4.0	6	3010	4	405.0	825.0	
1.0	2015-01-23 23:16:08	2015-01-24 00:06:16	"a914ecef9c12ffdb9bede64bb703d8...	"fast"	4.0	5	1836	3	300.0	399.0	
1.0	2015-02-01 12:48:15	2015-02-01 13:53:22	"c81e155d85dae5430a8cee6f2242e8...	"sandwich"	1.0	1	1175	1	535.0	535.0	
1.0	2015-02-08 13:54:33	2015-02-08 14:31:41	"c81e155d85dae5430a8cee6f2242e8...	"sandwich"	1.0	4	2605	4	425.0	750.0	

```
In [43]: cols=[
        'order_protocol',
        'total_items',
        'subtotal',
        'num_distinct_items',
        'min_item_price',
        'max_item_price',
        'total_onshift_partners',
    ]
```



```
'total_busy_partners',
'total_outstanding_orders',
'created_at_year',
'created_at_month',
'created_at_day',
'created_at_hour',
'created_at_minute',
'actual_delivery_time_year',
'actual_delivery_time_month',
'actual_delivery_time_day',
'actual_delivery_time_hour',
'actual_delivery_time_minute'
]
```

```
In [44]: # scaler = StandardScaler()
# scaled_features = scaler.fit_transform(df[cols].to_numpy())
knn_imputer = KNNImputer(n_neighbors=3)
imputed_features = knn_imputer.fit_transform(df[cols].to_numpy())
```

```
In [45]: # imputed_features = scaler.inverse_transform(imputed_features)
imputed_features = pl.DataFrame(imputed_features)
imputed_features.columns = cols
imputed_features
```

Out[45]: shape: (177_544, 19)

order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshift_partners	total_busy_partners	total_outstanding_orders	created_at_year	created_at_month	created_at_d
f64	f64	f64	f64	f64	f64	f64	f64	f64	f64	f64	f
1.0	4.0	3441.0	4.0	557.0	1239.0	33.0	14.0	21.0	2015.0	2.0	f
2.0	1.0	1900.0	1.0	1400.0	1400.0	1.0	2.0	2.0	2015.0	2.0	10
1.0	1.0	1900.0	1.0	1900.0	1900.0	1.0	0.0	0.0	2015.0	1.0	20
1.0	6.0	6900.0	5.0	600.0	1800.0	1.0	1.0	2.0	2015.0	2.0	30
1.0	3.0	3900.0	3.0	1100.0	1600.0	6.0	6.0	9.0	2015.0	2.0	10
...	...	...	...	...	...	...	...	...	...	...	...
4.0	3.0	1389.0	3.0	345.0	649.0	17.0	17.0	23.0	2015.0	2.0	10
4.0	6.0	3010.0	4.0	405.0	825.0	12.0	11.0	14.0	2015.0	2.0	10
4.0	5.0	1836.0	3.0	300.0	399.0	39.0	41.0	40.0	2015.0	1.0	20
1.0	1.0	1175.0	1.0	535.0	535.0	7.0	7.0	12.0	2015.0	2.0	...
1.0	4.0	2605.0	4.0	425.0	750.0	20.0	20.0	23.0	2015.0	2.0	...

```
In [46]: df=df.with_columns(
    pl.when(pl.col("order_protocol").is_null())
      .then(imputed_features["order_protocol"])
      .otherwise(pl.col("order_protocol"))
      .cast(pl.Int8)
      .alias("order_protocol")
)
```

```
In [47]: df.write_parquet("../data/cleaned/dataset.parquet")
```

Feature Generation

```
In [54]: df=pl.read_parquet("../data/cleaned/dataset.parquet")
```

```
In [55]: df[["created_at"].max(), df[["created_at"].min()]
```

```
Out[55]: (datetime.datetime(2015, 2, 18, 0, 30, 44),
datetime.datetime(2015, 1, 21, 9, 52, 3))
```

```
In [56]: df=df.with_columns(
    pl.col("created_at").dt.strftime("%b").alias("created_at_month_name"),
    pl.col("actual_delivery_time").dt.strftime("%b").alias("actual_delivery_time_month_name"),
    pl.col("created_at").dt.weekday().alias("created_at_day_of_week"),
    pl.col("actual_delivery_time").dt.weekday().alias("actual_delivery_time_day_of_week"),
    pl.col("created_at").dt.strftime("%a").alias("created_at_day_name"),
    pl.col("actual_delivery_time").dt.strftime("%a").alias("actual_delivery_time_day_name"),
    pl.col("created_at").dt.week().alias("created_at_week_number"),
    pl.col("actual_delivery_time").dt.week().alias("actual_delivery_time_week_number"),
    ((pl.col("created_at").dt.day() - 1) // 7 + 1).cast(pl.Int16).alias("created_at_week_of_month"),
    ((pl.col("actual_delivery_time").dt.day() - 1) // 7 + 1).cast(pl.Int16).alias("actual_delivery_time_week_of_month")
)
```

```
In [57]: df=df.with_columns(
    ((pl.col("actual_delivery_time") - pl.col("created_at")).dt.total_seconds())
      .alias("delivery_time_seconds"),
    ((pl.col("actual_delivery_time") - pl.col("created_at")).dt.total_minutes())
      .alias("delivery_time_minutes"),
)
```

```
In [58]: df.with_columns(
    pl.col("market_id").cast(pl.Int8).alias("market_id"),
    pl.col("total_items").cast(pl.Int16).alias("total_items"),
    pl.col("subtotal").cast(pl.Int16).alias("subtotal"),
    pl.col("num_distinct_items").cast(pl.Int16).alias("num_distinct_items"),
    pl.col("min_item_price").cast(pl.Int16).alias("min_item_price"),
    pl.col("max_item_price").cast(pl.Int16).alias("max_item_price"),
)
```

Out [58]: shape: (177_544, 38)

market_id	created_at	actual_delivery_time	store_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshi
i8	datetime[μs]	datetime[μs]	str	str	i8	i16	i16	i16	i16	i16	
1	2015-02-06 16:54:17	2015-02-06 17:57:16	"df263d996281d984952c07998dc543...	"american"	1	4	3441	4	557	1239	
2	2015-02-10 16:19:25	2015-02-10 17:26:29	"f0ade77b43923b38237db569b016ba...	"mexican"	2	1	1900	1	1400	1400	
3	2015-01-22 15:09:28	2015-01-22 15:39:09	"f0ade77b43923b38237db569b016ba...	"other"	1	1	1900	1	1900	1900	
3	2015-02-03 15:51:45	2015-02-03 16:43:00	"f0ade77b43923b38237db569b016ba...	"other"	1	6	6900	5	600	1800	
3	2015-02-14 21:10:36	2015-02-14 21:50:26	"f0ade77b43923b38237db569b016ba...	"other"	1	3	3900	3	1100	1600	
...	...	...	...	...	...	...	...	...	...	...	
1	2015-02-16 18:49:41	2015-02-16 19:54:48	"a914ecef9c12ffdb9bede64bb703d8...	"fast"	4	3	1389	3	345	649	
1	2015-02-12 18:31:59	2015-02-12 19:28:22	"a914ecef9c12ffdb9bede64bb703d8...	"fast"	4	6	3010	4	405	825	
1	2015-01-23 23:16:08	2015-01-24 00:06:16	"a914ecef9c12ffdb9bede64bb703d8...	"fast"	4	5	1836	3	300	399	
1	2015-02-01 12:48:15	2015-02-01 13:53:22	"c81e155d85dae5430a8cee6f2242e8...	"sandwich"	1	1	1175	1	535	535	
1	2015-02-08 13:54:33	2015-02-08 14:31:41	"c81e155d85dae5430a8cee6f2242e8...	"sandwich"	1	4	2605	4	425	750	

In [59]: df.write_parquet("../data/cleaned/dataset.parquet")